

Subgraph Analysis of the Apache Groovy Base Snapshot

Empirical Findings Across Seven Code Representation Families
for the KG-Commit Knowledge Graph

KG-Commit Research Project

June 2026

Contents

1	Background and Setup	3
1.1	What is KG-Commit?	3
1.2	The Base Snapshot: Apache Groovy	3
1.3	What is a Subgraph Family?	3
1.4	Project Structure Overview	4
1.5	How the Analysis Was Performed	4
2	Family 1 Token / Lexical Subgraph	5
2.1	What it is	5
2.2	Empirical Findings	5
2.3	Visualisations	5
2.4	Role in the KG	5
3	Family 2 Abstract Syntax Tree (AST)	7
3.1	What it is	7
3.2	Empirical Findings	7
3.3	Visualisations	8
3.4	Role in the KG	8
4	Family 3 Control Flow Graph (CFG)	10
4.1	What it is	10
4.2	Empirical Findings	10
4.3	Visualisations	11
4.4	Role in the KG	11
5	Family 4 Data Flow Graph (DFG)	12
5.1	What it is	12
5.2	Empirical Findings	12
5.3	Visualisations	12
5.4	Role in the KG	12

6	Family 5 Code Property Graph (CPG / PDG)	13
6.1	What it is	13
6.2	Empirical Findings	13
6.3	Visualisations	13
6.4	Role in the KG	13
7	Family 6 Semantic Analysis	15
7.1	What it is	15
7.2	Empirical Findings	15
7.3	Visualisations	16
7.4	Role in the KG	16
8	Family 7 Program Slicing	17
8.1	What it is	17
8.2	Empirical Findings	17
8.3	Visualisations	17
8.4	Role in the KG	18
9	Comparative Analysis and KG Recommendations	19
9.1	What each family uniquely contributes	19
9.2	Subsumption relationships	19
9.3	The codebase portrait	19
9.4	Recommended subgraph combinations	20
10	Output Files Reference	21
10.1	CSV Feature Tables (outputs/*.csv)	21
10.2	JSON Data Files (outputs/*.json)	23
10.3	Pickle Graph Files (outputs/*_example_graphs/*.pkl)	24
10.4	Build Log Files (outputs/*_build_log.txt)	25

1 Background and Setup

1.1 What is KG-Commit?

KG-Commit is a research project that models the evolution of a software repository as a **Knowledge Graph (KG)**. The core idea is: instead of treating code changes (commits) as raw text diffs, we represent the entire codebase as a graph of interconnected entities files, classes, methods, tokens and then model how that graph changes over time.

A **Knowledge Graph** is a structured representation where:

- **Nodes** represent things (a method, a class, a file, a token)
- **Edges** represent relationships between them (“method A calls method B”, “class C is defined in file D”, “token X appears after token Y”)

By embedding this graph into a vector space, a machine learning model can learn to predict things like: *Which methods are likely to change together?* *Where are bugs most likely to appear?* *What is the impact radius of a given commit?*

1.2 The Base Snapshot: Apache Groovy

The analysis in this document is performed on the **first Java-language commit** of the Apache Groovy project, identified by commit hash 408b29851d7b. This is the earliest stable snapshot of the codebase from which all subsequent evolution begins.

Property	Value
Repository	Apache Groovy (GitHub)
Snapshot	First Java commit (408b29851d7b)
Java source files	120
Concrete methods analysed	996
Abstract / interface methods	excluded (no body to analyse)

1.3 What is a Subgraph Family?

The KG contains methods as nodes. A **subgraph family** is a way of expanding each method node into a richer internal graph – a mini-graph that represents some aspect of what the method does. Seven such families were implemented and analysed:

1. **Token / Lexical** the raw sequence of code tokens
2. **Abstract Syntax Tree (AST)** the grammatical structure
3. **Control Flow Graph (CFG)** the execution paths
4. **Data Flow Graph (DFG)** how data moves through the code
5. **Code Property Graph (CPG)** control + data dependence unified
6. **Semantic Analysis** behavioral properties and risk
7. **Program Slicing** what influences or is influenced by each statement

Each family answers a different question about a method. Together, they give a multi-perspective representation that is far richer than any single view.

1.4 Project Structure Overview

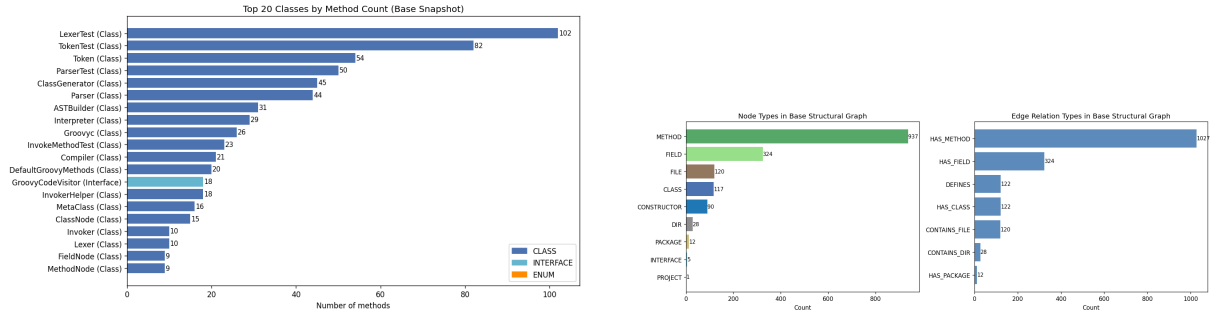


Figure 1: Left: the 20 largest classes by method count. Right: distribution of node and edge types in the base KG.

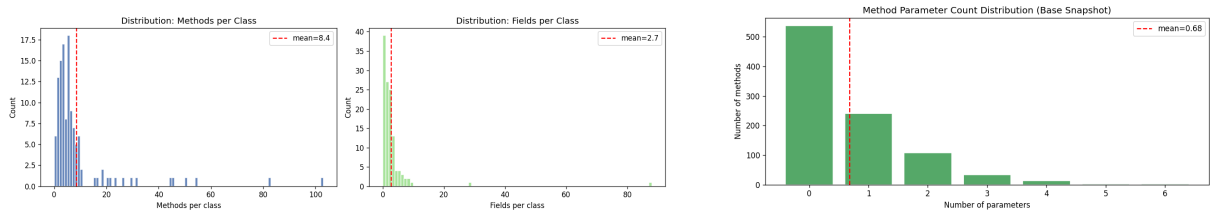


Figure 2: Left: distribution of methods and fields per class. Right: method parameter-count distribution.

1.5 How the Analysis Was Performed

Each family was computed **offline** parsed and extracted outside the notebook using a dedicated Python script (`build_X_features.py`). The notebook then loads the precomputed results for analysis and visualisation. This was necessary because the `javalang` parser is slow and can get stuck on certain files; each file was processed in a separate subprocess with a 45-second timeout so that pathological inputs could not block the entire analysis.

Every family produced:

- A **CSV file** with one row per method (the deliverable feature table)
- A **JSON file** of typed rows for the notebook
- An **aggregates file** with project-level statistics
- A small set of **example graphs** (pickled NetworkX objects) for visualisation

2 Family 1 Token / Lexical Subgraph

2.1 What it is

Before any grammar analysis, source code is just a sequence of **tokens** the atomic building blocks of the language, produced by the lexer (the first stage of a compiler). There are five token types:

Type	Examples	Role
Keyword	<code>if, return, for, class, new</code>	Language control words
Identifier	<code>myVar, getName, Groovy</code>	User-defined names
Separator	<code>{ } () ; ,</code>	Structural punctuation
Operator	<code>= + == != &&</code>	Computations / comparisons
Literal	<code>42 "hello" true null</code>	Raw values

The Family 1 subgraph connects these tokens in sequence: each token is a node, and there is a directed edge from each token to the next one. This produces a simple **token-chain graph** for each method.

2.2 Empirical Findings

Token counts (996 methods):

- Minimum: **2** tokens Median: **16** tokens Mean: **39.2** tokens Maximum: **2,473** tokens
- 90th percentile: 93 tokens meaning 10% of methods have more than 93 tokens
- The maximum (2,473) belongs to the Lexer's `nextToken` method a giant switch statement that handles every character in the Groovy language

The most revealing finding is the **composition of token types**:

Token type	Average share	Interpretation
Separator () { } ; ,	57.9%	Java boilerplate structure
Identifier (names)	25.8%	Variable/method/class names
Keyword	7.8%	Control constructs (<code>if/for/return</code>)
Literal	4.8%	Hard-coded values
Operator	3.6%	Arithmetic / comparisons

Over half of every method is punctuation. This is a property of Java's verbose syntax. The low keyword (7.8%) and operator (3.6%) rates confirm independently what the CFG analysis found: this codebase is predominantly straight-line code with few branches and computations.

Token diversity: The median method uses only **4 distinct token types** (out of a possible 10 fine-grained types). The type entropy (median **1.0 bits**) is low, indicating that most methods are built from a very narrow vocabulary.

2.3 Visualisations

2.4 Role in the KG

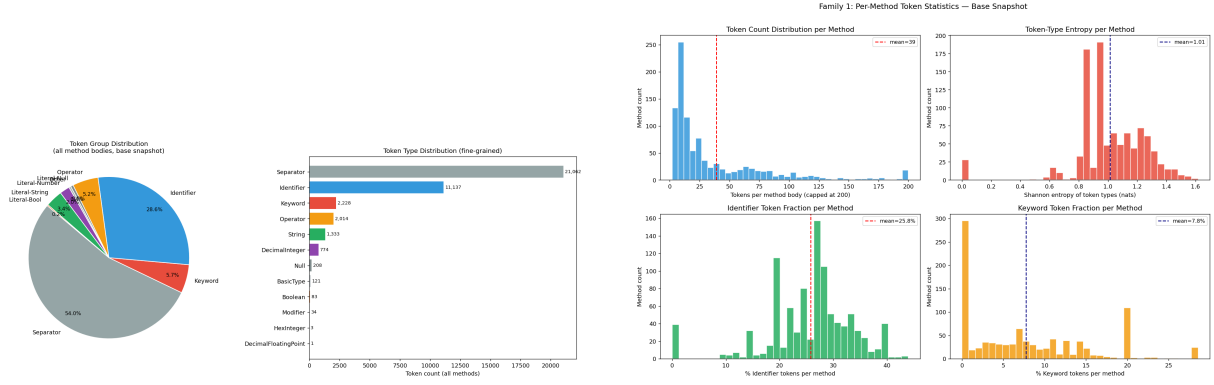


Figure 3: Left: token-type composition across all methods. Right: per-method token statistics (count, diversity, entropy).

Strengths: Cheapest to compute (no parsing, only lexing). Captures surface-level style, vocabulary, and structure. Directly compatible with token-level language models (CodeBERT, etc.). Good for clone detection and authorship fingerprinting.

Limitation: Tokens carry no meaning. The sequences $x = y + z$ and $a = b + c$ are structurally identical but token-level models see them as different. The AST and deeper families resolve this by abstracting away names.

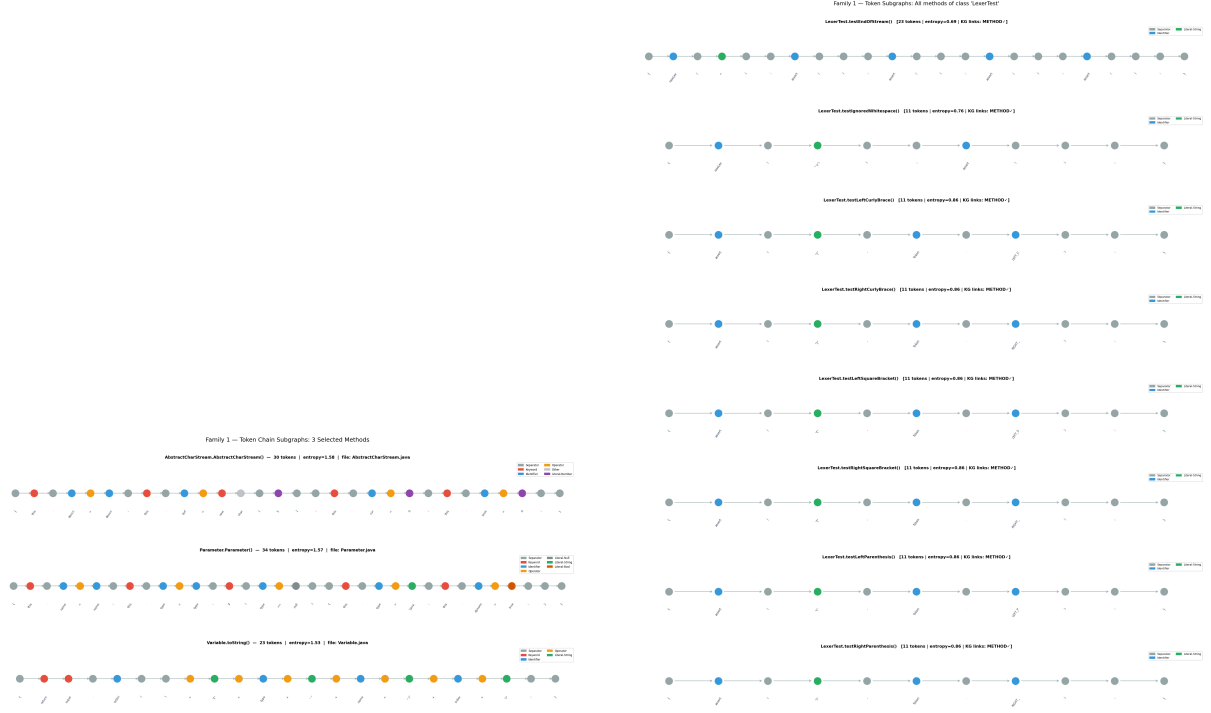


Figure 4: Left: example token-chain graphs for three selected methods. Right: token profiles for all methods in the `LexerTest` class.

3 Family 2 Abstract Syntax Tree (AST)

3.1 What it is

The **Abstract Syntax Tree** is the grammatical parse tree of a method. Every programming language has grammar rules (just like English has subject, verb, object). The AST is the tree diagram of how the code obeys those rules.

For example, the statement `return x + 1;` becomes a tree:

- `ReturnStatement` node
- $\hookrightarrow$ `BinaryOperation` node (+)
- $\hookrightarrow$ `MemberReference` node (x)
- $\hookrightarrow$ `Literal` node (1)

This tree captures the *grammatical shape* of the code, independent of the specific names used.

3.2 Empirical Findings

AST node counts (996 methods):

- Minimum: **3** nodes Median: **25** nodes Mean: **40.0** nodes Maximum: **1,902** nodes
- Total AST nodes across all methods: **39,848**

Tree depth:

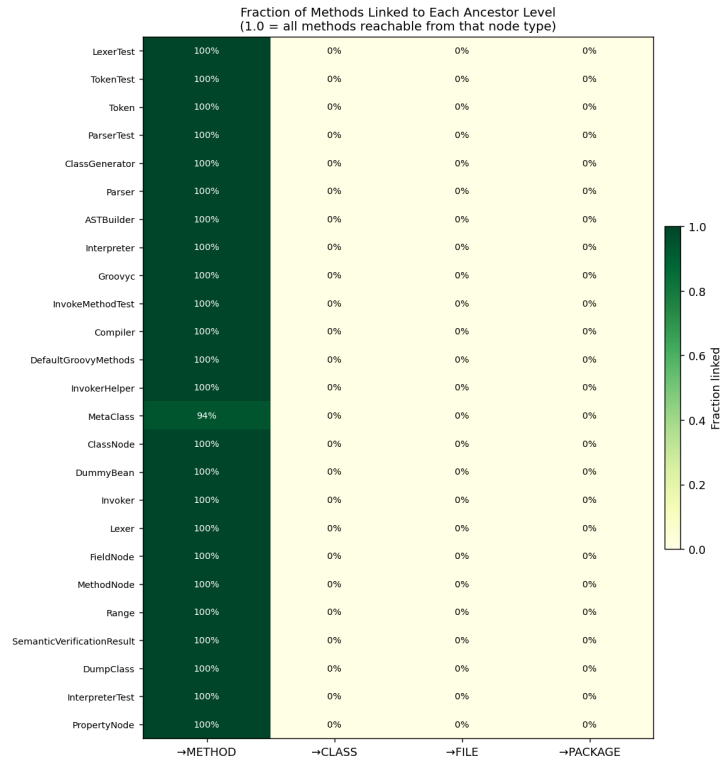


Figure 5: KG connectivity heatmap for Family 1 fraction of methods per class that are linked to each ancestor level (METHOD, CLASS, FILE, PKG) in the Knowledge Graph.

- Median: **5 levels** deep Mean: **5.2** Maximum: **20**
- A depth of 5 is typical: method → statement → expression → sub-expression → name

Type entropy (how varied the node types are):

- Median: **1.61 bits** Maximum: **2.20 bits**
- Methods near the entropy ceiling use a broad mix of constructs and are intrinsically harder to summarise or predict changes for

Most frequent node types across all methods:

Node type	Count	Meaning
Identifier	19,139	Any named reference
MemberReference	3,441	obj.field or variable read
MethodInvocation	3,327	A method call <code>foo()</code>
Literal	2,402	A constant value
ReferenceType	2,068	A type name like <code>String</code>

Identifiers account for nearly **48% of all AST nodes**, reflecting Java’s name-heavy style (every variable, type, and method call is a name).

3.3 Visualisations

3.4 Role in the KG

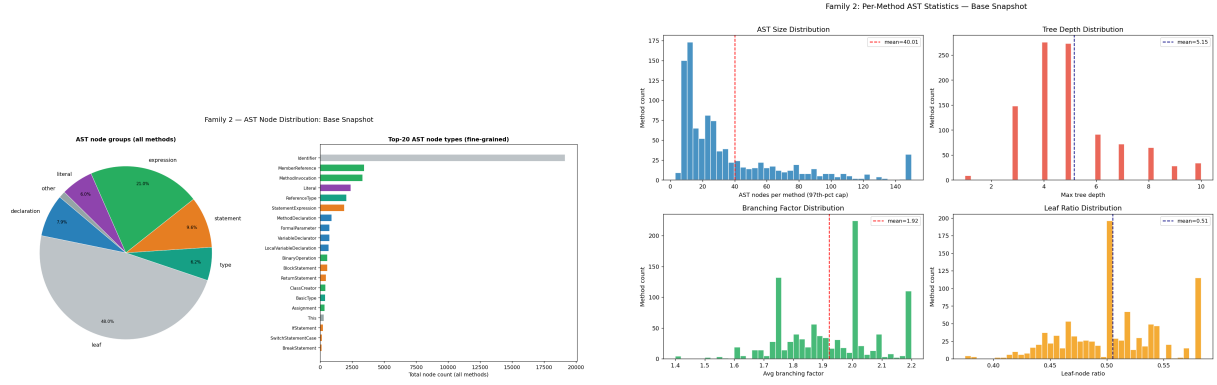


Figure 6: Left: distribution of AST node types across all methods. Right: per-method AST statistics (depth, node count, entropy).

Strengths: Captures syntactic structure independent of names. Two methods that do the same thing with different variable names have similar ASTs. Good for structural similarity, clone detection, and code-structure prediction tasks.

Compared to Family 1: The AST abstracts away the exact token sequence; $x = y + z$ and $a = b + c$ have the same AST shape. This makes it more generalisable.

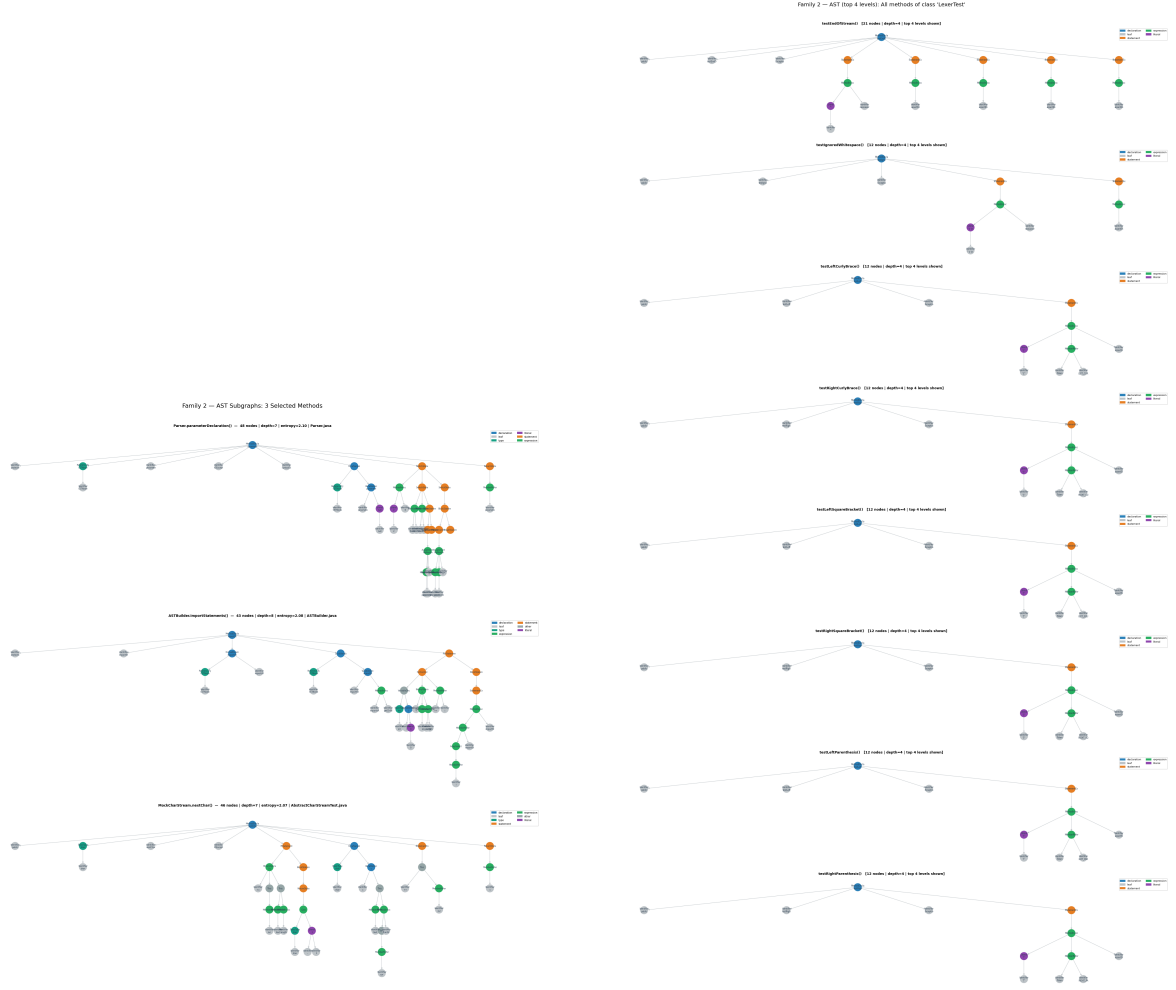


Figure 7: Left: example AST subgraphs for three selected methods. Right: AST profiles for all methods in the `LexerTest` class.

4 Family 3 Control Flow Graph (CFG)

4.1 What it is

When a program runs, it does not always execute statements in order. An `if` statement creates a branch: if the condition is true, go one way; if false, go another. A `for` loop goes back. The **Control Flow Graph** is the literal flowchart of a method – nodes are *basic blocks* (straight-line sequences of statements), and edges represent possible jumps between them.

The CFG also records the project-level **call graph**: which methods call which other methods.

4.2 Empirical Findings

Cyclomatic complexity (number of independent execution paths):

- Minimum: **1** Median: **1** Mean: **1.5** Maximum: **86**
- 90th percentile is only **2** meaning 90% of methods have at most one decision

point

- The maximum (**86**) is `Lexer.nextToken`: a parser method with 86 distinct cases

Loops and branching:

- Only **8.1%** of methods contain any loops
- Median loop count: **0** Median nesting depth: **0**
- 90% of methods have nesting depth ≤ 1

Call graph:

- **4,189** internal call edges (method calls another method in the same project)
- **106** distinct external callee names (calls to Java standard library or other external code)

Key insight: The codebase is **overwhelmingly linear**. The median cyclomatic complexity of 1 means the typical method has exactly one way to run – no branches, no loops. This is not unusual for an early snapshot: the complex control flow accumulates as the project matures. The 4,189 internal call edges are the primary source of inter-method dependency and are critical for predicting change impact.

4.3 Visualisations

4.4 Role in the KG

Strengths: The CFG enables reachability analysis (“can execution reach this statement from that one?”), dead-code detection, and inter-method change-impact analysis via the call graph. It is the foundation for Families 4 and 5.

Compared to Family 2 (AST): The CFG loses syntactic detail (it abstracts statements into blocks) but gains execution semantics. The AST knows that a node is an `IfStatement`; the CFG knows that this `if` creates a branch to two different continuations.

5 Family 4 Data Flow Graph (DFG)

5.1 What it is

Variables carry data. Every variable has:

- A **definition** (DEF): a point where it receives a value (assignment, parameter, loop variable, etc.)
- A **use** (USE): a point where its value is read

The **Data Flow Graph** draws an arrow from each DEF to every USE that the defined value can reach, following the actual execution paths in the CFG. This is called a *reaching-definitions* analysis.

5.2 Empirical Findings

Total across all 996 methods:

- DEF nodes: **1,709** USE nodes: **2,021** Def-use edges: **1,991**
- Average per method: **1.7 DEFs**, **2.0 USEs**, **2.0 edges**
- Maximum def-use edges in one method: **86**

Dead stores (unused definitions):

- **513 out of 1,709** definitions (30.0%) are never used
- A variable is defined but its value is never read within the method
- This is the highest-signal actionable finding across all families

Key insight: A **30% dead-store rate** is high. In a mature codebase, dead stores are typically < 5%. In this early snapshot of Apache Groovy, they reflect:

- Incomplete implementations (variable prepared for logic not yet written)
- Refactored code where old assignments were not cleaned up
- Defensive initialisation patterns common in early Java

This is a concrete, measurable signal of *code immaturity* that a KG embedding model should be able to learn from.

5.3 Visualisations

5.4 Role in the KG

Strengths: The DFG captures variable liveness and data dependencies within a method. It is essential for understanding *why* a change in one statement affects another. The dead-store rate is a direct quality signal that can be used to rank methods by maintenance need.

Compared to Family 3 (CFG): The CFG tells you *where* execution can go; the DFG tells you *what data* flows along those paths. They are complementary. Family 5 unifies them.

6 Family 5 Code Property Graph (CPG / PDG)

6.1 What it is

The **Code Property Graph** is the union of three graph layers in a single *multigraph* (a graph where multiple edges between the same pair of nodes are allowed, one per layer):

Layer	Full name	Meaning
CFG	Control Flow Graph	How execution moves
CDG	Control Dependence Graph	“This statement only runs because of that if ”
DDG	Data Dependence Graph	“This value was defined there and used here”

The CFG + CDG + DDG together form the **Program Dependence Graph (PDG)**, which is the most complete single-method code representation. It subsumes Families 3 and 4 entirely.

Important implementation note: This graph must be a **MultiDiGraph** (not a plain directed graph), because the same pair of statements can be connected by both a CFG edge and a CDG edge simultaneously. A plain directed graph would silently overwrite one edge with the other, destroying information.

6.2 Empirical Findings

Edge totals across all 996 methods:

- CFG edges: **5,227**
- CDG edges: **3,699**
- DDG edges: **1,914**
- **Total CPG edges: 10,840**

Per-method statistics (PDG edges = CDG + DDG):

- Median PDG edges: **3** Mean: **5.6** Maximum: **393**
- CDG:DDG ratio $\approx$ **2:1**

Key insight: The **2:1 CDG-to-DDG ratio** means that *control structure is the dominant source of dependence* in this codebase – most statements depend on whether a condition was true, not on what value a variable holds. This is consistent with the CFG finding (mostly linear code) and the DFG finding (few def-use chains): data flow is sparse, but even simple **if** statements create strong control dependences on their condition.

6.3 Visualisations

6.4 Role in the KG

Strengths: The CPG is the single most information-dense subgraph. It strictly contains the CFG and DFG as sub-layers. For any task requiring a *single* method-level subgraph – change impact, bug prediction, regression detection – the CPG provides the richest signal with the least loss of information.

Limitation: It is the most expensive to compute (requires a full reaching-definitions

worklist pass over the CFG). For very large methods it can become dense, making graph embedding harder. The maximum of 393 PDG edges in one method is a practical upper bound that remains manageable.

7 Family 6 Semantic Analysis

7.1 What it is

Families 15 all describe *structure*: what the code looks like or how control and data flow through it. Family 6 asks a different question: **what does this method actually *do* to the world?**

This is implemented as a source-level *approximation* no external tools (like Infer or Z3) are invoked. Instead, a lightweight pass over the AST infers:

- **Purity:** Does the method write to any field, do file I/O, or print output? If not, it is *pure* (like a mathematical function same inputs always give same outputs, no side effects).
- **NPE risk:** Does the method call a method on a variable (`x.something()`) without first checking if `x` is `null`? Such calls can crash at runtime with a `NullPointerException`.
- **Taint:** Does the method touch data that could come from an untrusted source (user input, files, network)?
- **Preconditions:** Does the method explicitly validate its inputs (`if (x == null) throw ..., Objects.requireNonNull, assertions`)?
- **Nullness:** Which variables carry null-risk at which points? Visualised as a *nullness subgraph* with VAR, DEREf, CHECK, and GUARD nodes.

7.2 Empirical Findings

Behavioral properties across 996 methods:

Property	Count	Rate	Interpretation
Pure (no side effects)	945	94.9%	Very functionally disciplined
Writes a field	27	2.7%	Mutates shared object state
Does I/O	28	2.8%	File, network, or print
Has NPE risk	154	15.5%	Potential crash point
Has taint source/sink	330	33.1%	Handles untrusted data
Has preconditions	19	1.9%	Explicit input validation

Key insights:

(1) **94.9% pure methods** is exceptionally high. Most methods are mathematical functions in disguise: they compute and return a value without touching the outside world. This suggests the codebase was written with functional discipline from the start.

(2) **15.5% NPE risk** with **0% annotation coverage** (`@Nullable/@NonNull`) means there are 154 methods that dereference variables without null guards, and the codebase has no annotation-based safety net to detect them. These are the highest-priority methods for a bug-prediction model to flag.

(3) **33.1% taint involvement** with only **1.9% preconditions** means that one-third of methods handle potentially untrusted data but almost none of them validate it. A large surface area for injection-style bugs.

7.3 Visualisations

7.4 Role in the KG

Strengths: Semantic features are *orthogonal* to all structural families. Two methods can have identical ASTs and CFGs but completely different semantic profiles (one pure, one side-effectful; one null-safe, one NPE-prone). This makes Family 6 indispensable for bug prediction, patch correctness classification, and security-relevant change detection.

Limitation: All findings are *approximations* a proper analysis would require whole-program points-to analysis, which is computationally expensive. The source-level proxy is conservative: it under-reports risk (some NPEs are missed) but rarely over-reports.

8 Family 7 Program Slicing

8.1 What it is

A **program slice** is the answer to the question: *if I change this one statement, what else in this method is affected?* Or conversely: *what other statements could have caused the value at this point to be what it is?*

Formally, given a *criterion* statement c :

- The **backward slice** of c is the set of all statements that may have influenced the value computed at c . It is found by following dependence edges (CDG + DDG from Family 5) *backwards* from c .
- The **forward slice** of c is the set of all statements that c may influence. It is found by following those same dependence edges *forwards*.

The key metric is **backward slice size**: how much of the method does a single statement depend on? A large backward slice means the statement is “deeply connected” to the rest of the method – a risky change point.

8.2 Empirical Findings

Backward-slice sizes across 996 methods:

- Minimum: **2** Median: **2** Mean: **2.9** Maximum: **83**
- 90th percentile: **4** most methods have very small slices
- Slice coverage (largest slice as fraction of method size): 76% of methods have coverage > 70%

Return-slice size (backward slice of the return/throw statement):

- Median: **0** Mean: **1.3** Maximum: **83**
- Half of all methods return a value that depends on nothing else in the method (trivial: `return x`)

Why is slice coverage so high (76% of methods > 70%)?

This is a direct consequence of the codebase’s linearity established in Family 3. In a *straight-line* method (no branches, no loops), every statement depends on every statement before it. If a method has 5 statements s_1, s_2, s_3, s_4, s_5 in sequence, and s_5 uses a variable set by s_1 , then $\text{backward-slice}(s_5) = \{s_1, s_2, s_3, s_4, s_5\}$ the entire method.

The *interesting* signal comes from the long tail: the outlier methods where slice coverage is *low*. Those are methods with **independent sub-computations** – computations that do not depend on each other. Their slice boundaries are natural *subgraph cut points* for decomposition, refactoring, or modular embedding.

8.3 Visualisations

Slice visualisation plots (backward-slice size distribution, slice coverage CDF, PDG with backward-slice highlighted) are generated by the notebook after running `build_slice_features.py`.

They are stored in `outputs/slice_example_graphs/` as pickle files and rendered interactively in the notebook.

8.4 Role in the KG

Strengths: Slicing operationalises the notion of *change scope*. If a commit modifies statement c , the forward slice of c is the set of statements that may now behave differently a programmatic definition of “regression risk”. This makes Family 7 directly applicable to:

- Predicting which test cases to re-run after a commit
- Ranking which other methods may be broken by a change
- Identifying tightly-coupled code regions

Compared to CPG (Family 5): The CPG *contains* the dependence edges; slicing is a *derived* computation over them. Family 7 does not add new graph structure but adds *scalar features* (slice sizes per statement and per method) that compress the most important structural signal into a form directly usable by downstream ML models.

9 Comparative Analysis and KG Recommendations

9.1 What each family uniquely contributes

Family	Name	Unique signal	Best KG tasks
1	Token	Surface vocabulary; token-type ratios	Clone detection, style similarity, LM pre-training
2	AST	Syntactic shape; node-type entropy	Structural similarity, grammar-level clone detection
3	CFG	Execution paths; call graph	Reachability, dead code, inter-method impact
4	DFG	Def-use chains; dead stores (30%)	Variable liveness, data-dependency prediction
5	CPG	All dependences unified; dominant signal	General-purpose change impact, KGE embedding
6	Semantic	Purity, NPE risk (15.5%), taint (33.1%)	Bug prediction, security classification
7	Slicing	Change scope; slice coverage	Regression scope, test prioritisation

9.2 Subsumption relationships

Some families contain others as strict subsets:

- **CPG $\supset$ CFG + DFG**: Family 5 strictly contains Families 3 and 4. If you use the CPG, adding CFG or DFG separately adds no new structural information.
- **Slicing $\leftarrow$ CPG**: Family 7 is derived from the CPG’s dependence edges. Slicing features are a *compression* of CPG structure into scalars.
- **AST $\not\supset$ Tokens**: The AST abstracts away the token sequence but loses ordering information. They are complementary, not hierarchical.
- **Semantic $\perp$ all structural**: Behavioral properties (purity, NPE risk) are orthogonal to all graph-structural families.

9.3 The codebase portrait

Reading across all seven families together, a consistent portrait emerges of this early Apache Groovy snapshot:

1. **Structurally simple, long-tailed.** 90% of methods are short, linear, and low-complexity. The top 10% carry nearly all the structural richness. Any KG embedding must handle this heavy-tail distribution.
2. **Functionally disciplined.** 94.9% pure methods is unusually high. The codebase was written with a functional style where methods compute results without side effects.

3. **Early-stage code quality.** The 30% dead-store rate and the 15.5% NPE-risk rate with 0% annotation coverage are measurable signs of an immature codebase. These are exactly the kinds of signals a KG-based bug predictor should learn to recognise in early commits and track as they are fixed in later ones.
4. **Control-dominated dependencies.** The 2:1 CDG-to-DDG ratio means that branching logic (not data computation) is the primary source of inter-statement dependence. Embeddings that emphasise control structure will capture more signal than those focused on data flow for this codebase.
5. **High taint surface, low validation.** One-third of methods handle potentially untrusted data but barely 2% validate inputs. This is a security-relevant signal that persists into the project’s evolution and is directly trackable via Family 6’s taint features.

9.4 Recommended subgraph combinations

Use case	Recommended families	Rationale
General KG embedding	CPG (5) + AST (2)	CPG carries dependence signal; AST adds orthogonal shape signal
Bug / NPE prediction	CPG (5) + Semantic (6)	Dependence structure + behavioral risk
Change impact	CPG (5) + Slicing (7)	Unified dependence + quantified scope
Code similarity	AST (2) + Token (1)	Shape + vocabulary
Security audit	Semantic (6) + DFG (4)	Taint + data-flow paths
Full representation	All 7 families	Maximum expressiveness; use dimensionality reduction

Single-family recommendation: If only one subgraph family can be used, the **CPG (Family 5)** is the strongest choice. It strictly subsumes CFG and DFG, its 10,840 edges across the project capture the richest structural signal, and it directly supports both change impact analysis and slice computation.

Two-family recommendation: Add the **AST (Family 2)**. The AST’s syntactic shape signal is orthogonal to all dependence-based families – it captures *what the code looks like*, while the CPG captures *how its parts depend on each other*. This combination provides the highest-value two-family coverage for the KG-Commit downstream tasks.

10 Output Files Reference

All outputs are written to the `outputs/` directory at the project root. Every family produces three types of files: a CSV feature table, JSON data files, and pickle graph files.

10.1 CSV Feature Tables (`outputs/*.csv`)

Each CSV has one row per method. The first five columns are always the same identity fields: `method_id` (unique string `file::class::method@line`), `file`, `class`, `method`, `kind` (method or constructor). The last four columns are always KG connectivity flags: `has_method_link`, `has_class_ancestor`, `has_file_ancestor`, `has_pkg_ancestor` whether the method node in the KG is reachable from its parent class, file, and package nodes. The middle columns are family-specific features:

File	Family-specific columns
<code>token_subgraph_stats.csv</code>	<code>n_tokens</code> : total token count. <code>n_token_types</code> : number of distinct token types used. <code>n_token_groups</code> : distinct high-level groups (keyword/id/sep/op/lit). <code>type_entropy</code> : Shannon entropy of token-type distribution. <code>group_entropy</code> : entropy of group distribution. <code>density</code> : edge density of the token-chain graph. <code>pct_keyword</code> , <code>pct_identifier</code> , <code>pct_separator</code> , <code>pct_operator</code> , <code>pct_literal</code> : fraction of each token type.
<code>ast_subgraph_stats.csv</code>	<code>n_ast_nodes</code> : total AST nodes in the method. <code>n_edges</code> : AST edges (always = $n_nodes - 1$ for a tree). <code>max_depth</code> : deepest path from method root to leaf. <code>avg_depth</code> : average depth of all nodes. <code>avg_branching</code> : average number of children per non-leaf node. <code>n_node_types</code> , <code>n_groups</code> : type and group vocabulary sizes. <code>type_entropy</code> , <code>group_entropy</code> : structural variety. <code>leaf_ratio</code> : fraction of nodes that are leaves. <code>pct_declaration</code> , <code>pct_statement</code> , <code>pct_expression</code> , <code>pct_literal</code> , <code>pct_leaf</code> : fractions of each node category.

continues...

File	Family-specific columns
<code>cfg_subgraph_stats.csv</code>	<code>n_cfg_nodes</code> : number of basic blocks. <code>n_cfg_edges</code> : number of control-flow edges. <code>cyclomatic</code> : McCabe cyclomatic complexity ($= \text{edges} - \text{nodes} + 2$). <code>n_decision_points</code> : number of branch-creating statements. <code>n_branches</code> , <code>n_loops</code> , <code>n_switch</code> : counts by type. <code>n_returns</code> , <code>n_throws</code> , <code>n_breaks</code> , <code>n_continues</code> : exit counts. <code>n_catch</code> : exception-handler blocks. <code>has_try</code> : whether the method contains a try block. <code>n_back_edges</code> : back edges (one per loop body). <code>max_nesting</code> : maximum loop/branch nesting depth. <code>n_call_sites</code> : number of method invocations. <code>kind_entropy</code> : entropy of block-kind distribution.
<code>dfg_subgraph_stats.csv</code>	<code>n_def_nodes</code> : definition (DEF) nodes. <code>n_use_nodes</code> : use (USE) nodes. <code>n_du_edges</code> : reaching-definition (DEF→USE) edges. <code>n_variables</code> : distinct variable names. <code>n_params</code> : parameter count. <code>n_unused_defs</code> : definitions whose value is never read (dead stores). <code>n_undef_uses</code> : uses with no reachable definition (possibly a field read). <code>max_uses_per_def</code> : maximum fan-out of any single definition. <code>avg_uses_per_def</code> : average fan-out. <code>du_density</code> : edge density of the def-use graph. <code>var_entropy</code> : entropy of per-variable def/use distribution.
<code>cpg_subgraph_stats.csv</code>	<code>n_cpg_nodes</code> : total nodes in the CPG. <code>n_cfg_edges</code> : control-flow layer edge count. <code>n_cdg_edges</code> : control-dependence layer edge count. <code>n_ddg_edges</code> : data-dependence layer edge count. <code>n_total_edges</code> : sum of all three layers. <code>pdg_edges</code> : dependence-only edge count ($= n_cdg + n_ddg$). <code>cyclomatic</code> : cross-check with Family 3 (must match). <code>n_predicates</code> : number of predicate (branching) nodes. <code>max_nesting</code> : maximum control-nesting depth. <code>edges_per_node</code> : total edge density. <code>kind_entropy</code> : entropy of node-kind distribution.

continues...

File	Family-specific columns
<code>sem_subgraph_stats.csv</code>	<code>n_params</code> , <code>n_locals</code> : method signature info. <code>n_null_literals</code> , <code>n_null_returns</code> , <code>n_null_assigns</code> : null-related expressions. <code>n_null_checks</code> : explicit <code>== null</code> / <code>!= null</code> tests. <code>n_guards</code> : statements that guard against null propagation. <code>n_preconditions</code> : explicit input-validation calls (<code>requireNonNull</code> , assertions). <code>n_dereferences</code> : member accesses that could throw NPE. <code>n_unguarded_derefs</code> : dereferences without a preceding null check. <code>n_risk_vars</code> : distinct variables involved in NPE-risk dereferences. <code>n_nullable_annot</code> , <code>n_nonnull_annot</code> : annotation counts. <code>n_taint_sources</code> , <code>n_taint_sinks</code> : taint-flow endpoints. <code>writes_field</code> , <code>does_io</code> : boolean side-effect flags. <code>is_pure</code> : true if no field writes, no I/O, no static calls. <code>n_path_predicates</code> , <code>est_paths</code> : path-count proxy.
<code>slice_subgraph_stats.csv</code>	<code>n_pdg_nodes</code> : total PDG node count. <code>n_stmt_nodes</code> : statement nodes (excluding synthetic ENTRY/EXIT). <code>n_dep_edges</code> : dependence edges (CDG + DDG). <code>max_backward_slice</code> : largest backward-slice size over all criteria. <code>mean_backward_slice</code> : average backward-slice size. <code>max_forward_slice</code> : largest forward-slice size. <code>mean_forward_slice</code> : average forward-slice size. <code>return_slice_size</code> : backward slice of the method's return/throw statement. <code>slice_coverage</code> : $\text{max_backward_slice} / \text{n_stmt_nodes}$. <code>mean_slice_coverage</code> : average slice coverage across all criteria. <code>n_criteria</code> : number of statement criteria (equals <code>n_stmt_nodes</code>).

10.2 JSON Data Files (outputs/*.json)

***_rows.json**

One JSON object per method, containing the same fields as the corresponding CSV. Used by the notebook to load data with correct Python types (integers stay integers, booleans stay booleans CSV stores everything as strings). One file per family: `ast_rows.json`, `cfg_rows.json`, `dfg_rows.json`, `cpg_rows.json`, `sem_rows.json`, `slice_rows.json`.

***_aggregates.json**

Project-level summary statistics, pre-computed for fast notebook loading. Contents vary by family:

- `ast_aggregates.json`: `type_totals` (dict of AST node type $\rightarrow$ count), `group_totals`, `n_methods`, `ok_files`.

- `cfg_aggregates.json`: `edge_type_totals` (NEXT/TRUE/FALSE/LOOP_BACK/etc.), `n_internal_call_edges`, `n_external_call_names`.
- `dfg_aggregates.json`: `def_kind_totals` (param/decl/assign/update/forvar/catchvar).
- `cpg_aggregates.json`: `node_kind_totals`, layer-level edge totals.
- `sem_aggregates.json`: `signal_totals` (purity, NPE risk, taint, I/O, field-write counts).
- `slice_aggregates.json`: `bsize_sample` (sampled list of all backward-slice sizes, capped at 20,000 entries for the histogram), `ok_files`, `skipped_files`.

cfg_call_graph.json

The project-level inter-method call graph. Structure: `{method_id: {internal: [callee_id, ...], external: [name, ...]}}`. `internal` lists method IDs within the project; `external` lists unresolved external callee names (Java standard library, etc.). Contains 4,189 internal edges across 996 methods.

10.3 Pickle Graph Files (outputs/*_example_graphs/*.pkl)

Each example-graph directory contains NetworkX graph objects for a small selection of methods (~12–15 per family), chosen to be visually informative (moderate size, representative structure). These are the graphs rendered in the notebook visualisation cells.

ast_example_graphs/*.pkl

A single `networkx.DiGraph`. Each node has attributes: `label` (the node’s text representation), `type` (fine-grained javalang class name, e.g. `MethodInvocation`), `kind` (coarse group: `declaration/statement/expression/literal/leaf`). Edges represent parent → child relationships in the AST.

cfg_example_graphs/*.pkl

A `(G, entry)` tuple. `G` is a `networkx.DiGraph`; `entry` is the node ID of the method’s entry block. Node attributes: `kind` (`ENTRY/EXIT/IF/LOOP/etc.`), `label`. Edge attribute: `type` (`NEXT/TRUE/FALSE/LOOP_BACK/CASE/FALL/THROW/EXCEPTION/RETURN`).

dfg_example_graphs/*.pkl

A single `networkx.DiGraph`. DEF nodes have attributes: `kind` (`param/decl/assign/update/forvar/catchvar`), `var` (variable name), `line`. USE nodes have `kind=use`, `var`, `line`. Edges (DEF → USE) carry a `var` attribute. The graph layout is column-based: x = variable index, y = source line number.

cpg_example_graphs/*.pkl

A `(G, entry)` tuple. `G` is a `networkx.MultiDiGraph` (multigraph required because multiple edge types can connect the same pair of nodes). Each edge carries a `layer` attribute: `cfg` (control flow), `cdg` (control dependence), or `ddg` (data dependence). Node attributes: `kind`, `label`.

sem_example_graphs/*.pkl

A single `networkx.DiGraph` representing the *nullness subgraph* of the method. Node kinds: `VAR` (variable with null risk), `DEREF` (member access that could NPE), `CHECK` (null comparison), `GUARD` (statement that prevents null propagation). Edges represent `MAY_BE_NULL` risk flow from a variable to its dereference site.

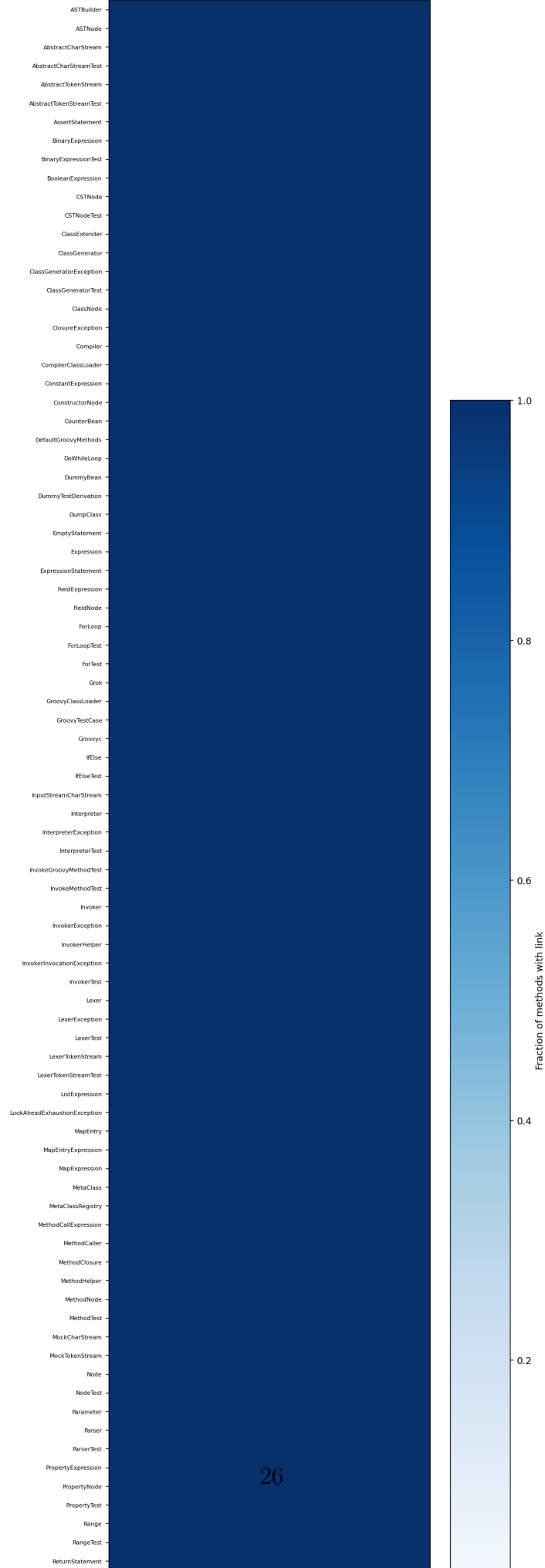
slice_example_graphs/*.pkl

A (G, entry, criterion, bslice) tuple. G is the same `networkx.MultiDiGraph` as in the CPG files (reused from `CPGBuilder`). `entry` is the entry node ID. `criterion` is the node ID of the statement with the largest backward slice. `bslice` is a sorted list of node IDs that are in the backward slice of `criterion` the set of all statements that may have influenced it.

10.4 Build Log Files (`outputs/*_build_log.txt`)

Each extractor writes a per-file processing log recording: which files were parsed successfully (with method count), which were killed by the 45-second subprocess timeout, and which failed with parse errors. These logs are useful for diagnosing why a particular file's methods are absent from the CSV.

Family 2: AST -> KG Connectivity (fraction of methods per class)



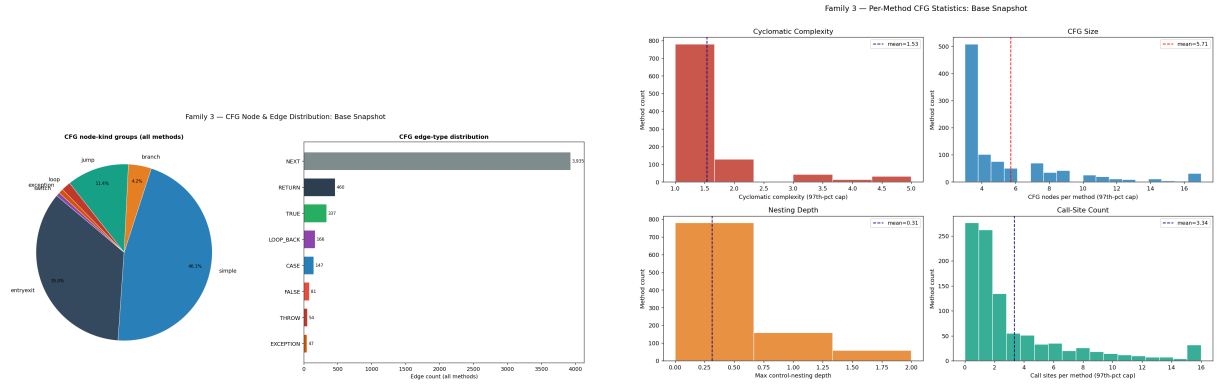


Figure 9: Left: cyclomatic complexity distribution across all methods. Right: per-method CFG statistics (blocks, branches, nesting, loops).

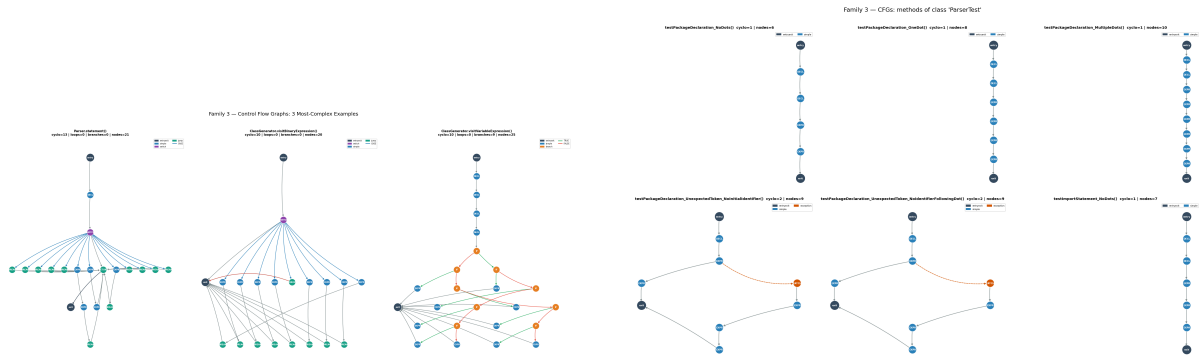


Figure 10: Left: example CFGs for three selected methods (edge colours: blue=NEXT, green=TRUE, red=FALSE, orange=LOOP_BACK). Right: CFG profiles for all methods in the `ParserTest` class.

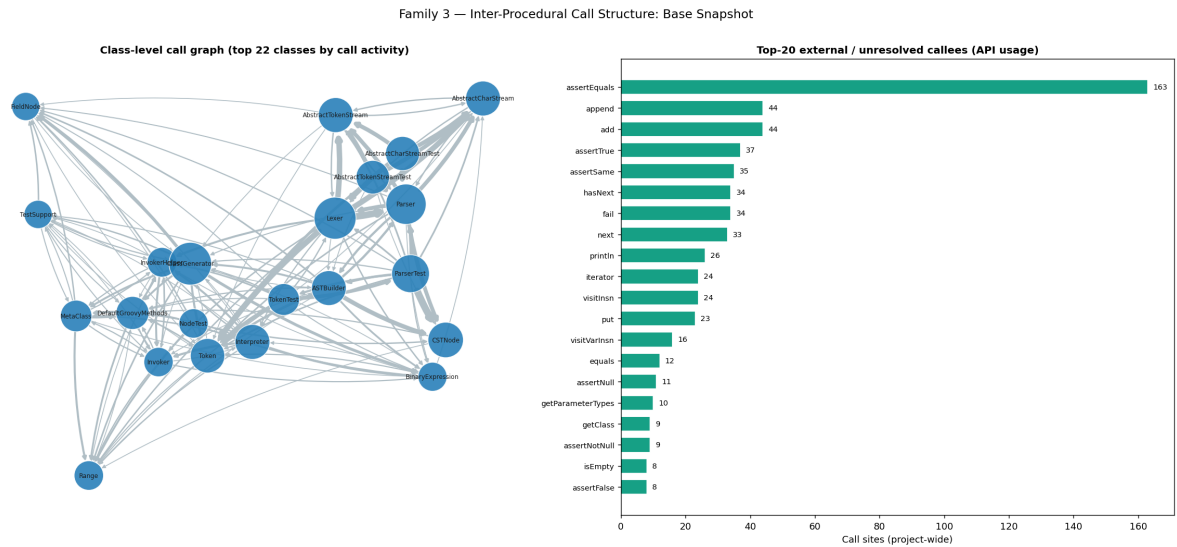


Figure 11: Project-level call graph. Each node is a method; directed edges represent calls from caller to callee. The 4,189 internal call edges make this the primary inter-method dependency structure in the base KG.

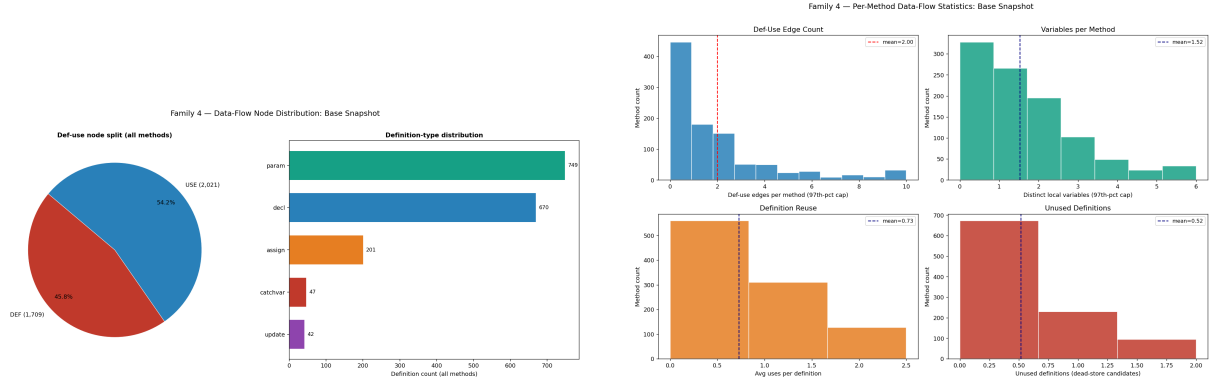


Figure 12: Left: distribution of def-use edges and dead-store rates. Right: per-method DFG statistics (DEF count, USE count, variable count, unused defs).

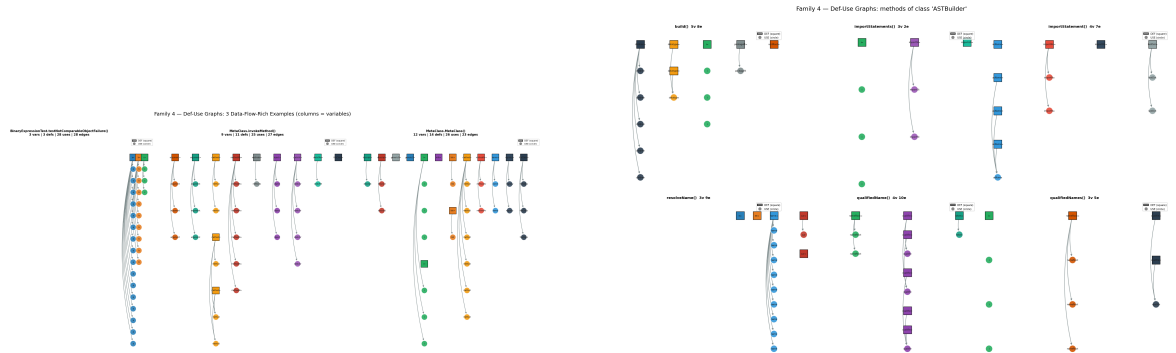


Figure 13: Left: example DFGs for three methods (columns = variables; rows = source lines; arrows = reaching definitions). Right: DFG profiles for all methods in the ASTBuilder class.

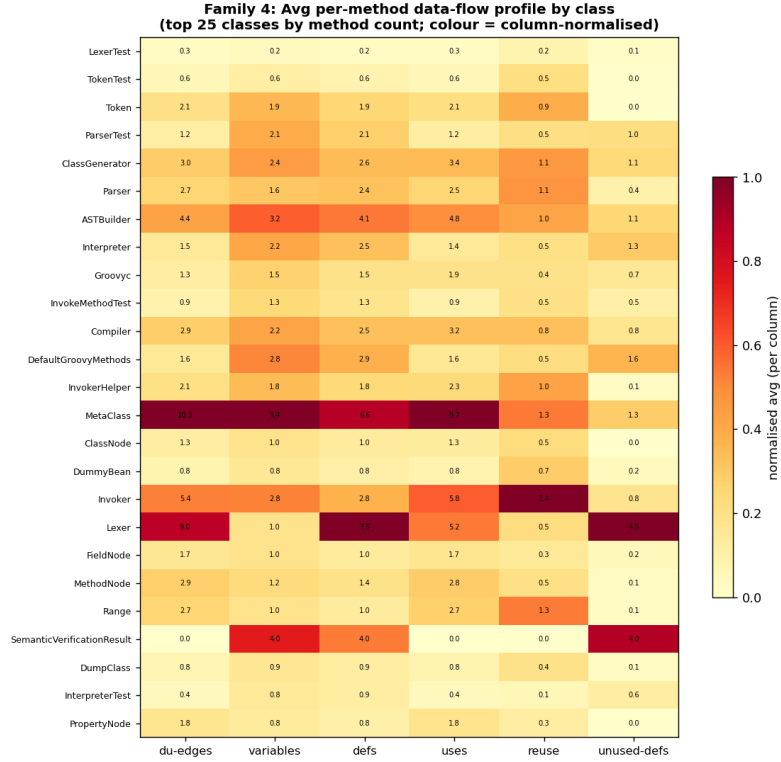


Figure 14: Heatmap of DFG features per class: each cell shows the average value of a DFG metric for all methods in a class, revealing which classes have the most data-flow complexity or the highest dead-store rates.

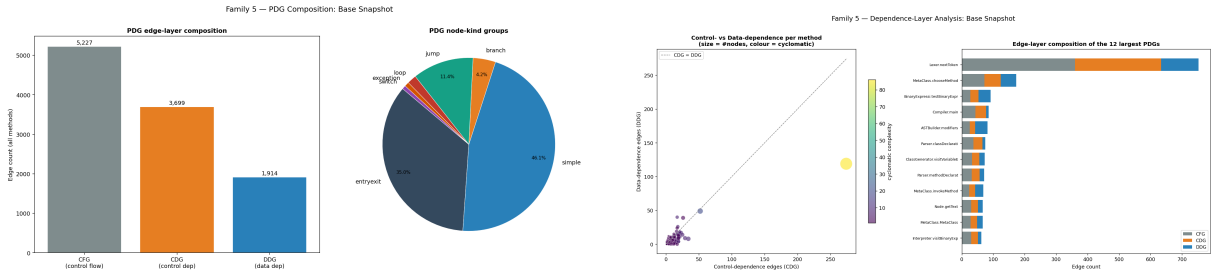


Figure 15: Left: breakdown of total CPG edges by layer (CFG/CDG/DDG) across all methods. Right: per-method analysis of each layer's contribution the 2:1 CDG-to-DDG ratio is visible here.

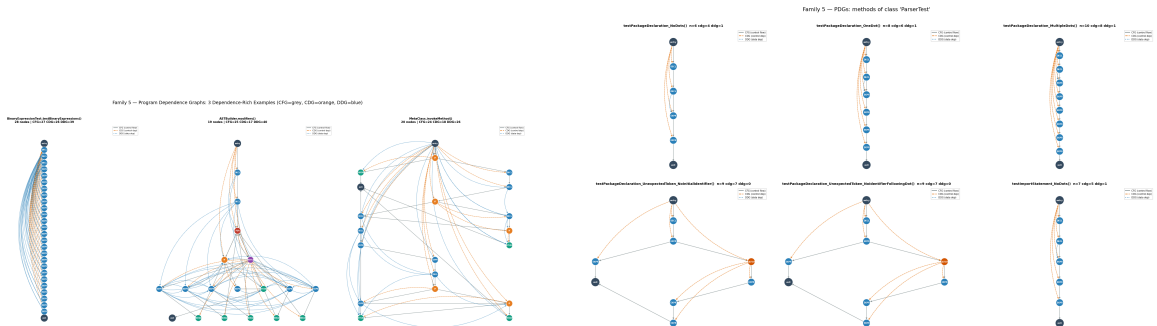


Figure 16: Left: example CPG multigraphs for three methods (grey=CFG, dashed=CDG, blue=DDG). Right: CPG profiles for all methods in the `ParserTest` class.

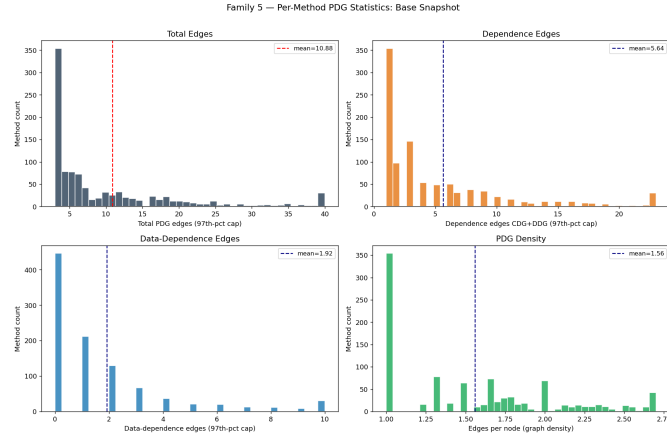


Figure 17: Per-method CPG statistics: node counts, edge counts per layer, cyclomatic complexity, and edges-per-node density.

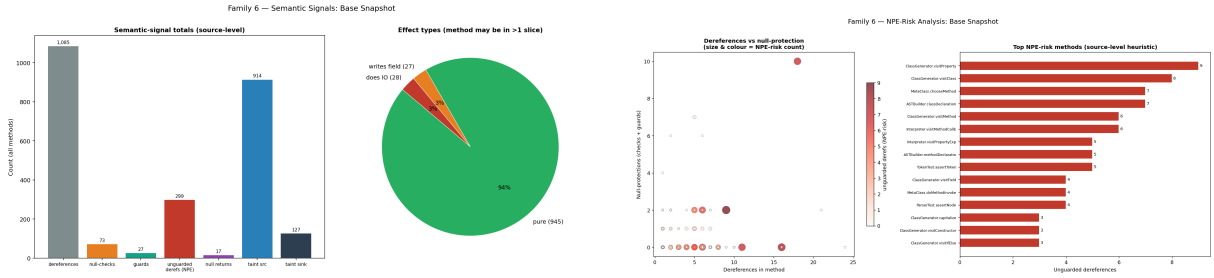


Figure 18: Left: prevalence of each semantic signal across all methods (purity, taint, I/O, field writes, preconditions). Right: NPE-risk analysis distribution of unguarded dereference counts per method and the methods most at risk.

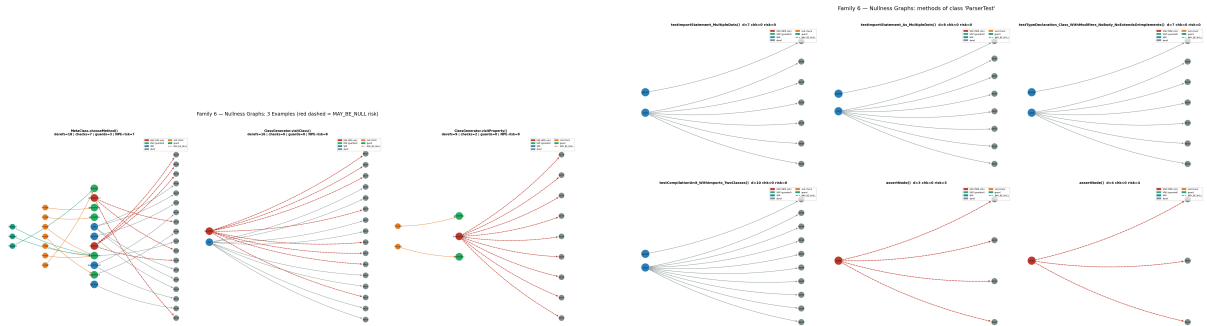


Figure 19: Left: example nullness subgraphs for three methods (green=VAR, orange=DEREF, blue=CHECK, red=GUARD nodes; dashed=MAY_BE_NULL risk edges). Right: semantic profiles for all methods in the `ParserTest` class.

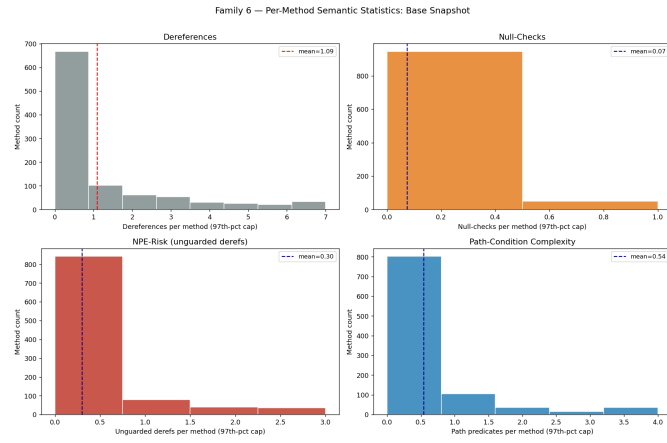


Figure 20: Per-method semantic statistics: deref counts, null-check counts, guard counts, taint source/sink flags, and purity flags.